



CLASS 0 · STUDENTS + FAMILIES · IN PERSON

00-01 MIN

From an idea to something that works.

Learning Python in the age of AI

Bo Shang · Merit Point Academy

60 minutes · No coding experience needed

Think together. Try a rule. Find a counterexample.



START WITH YOUR QUESTION

01–03 MIN

If AI can write code, what should you learn?

Take 20 seconds. Choose one.

A · Create

Build something you wish existed.

B · Understand

Find out how a digital system works.

C · Judge

Decide whether a result deserves
your trust.

Students answer first. Parents: what would you add?



My first steps: words became actions.

PRIMARY SCHOOL

BASIC

I began learning to give a computer instructions.

AFTER THE GAOKAO

Visual Basic

Visual interfaces connected code with buttons and events.

UNDERGRADUATE

A VB lottery program

I built a program for a lucky draw.

Text → interface → a tool people could use



Then code reached the physical world.

HARDWARE

Microcontroller programming

Code could read inputs and control outputs.

BUILDING SYSTEMS

Electronic design competition

Programming became part of an electronic system.

SENSING + MOVING

Freescale intelligent car competition

Software met sensors, motors and movement.

A command on a screen can become an action in the world.



From programming to systems research.

International Aerial Robotics Competition

Robotics brings sensing, computing and action together.

Mechatronics

Mechanical systems + electronics + computing.

Automation science

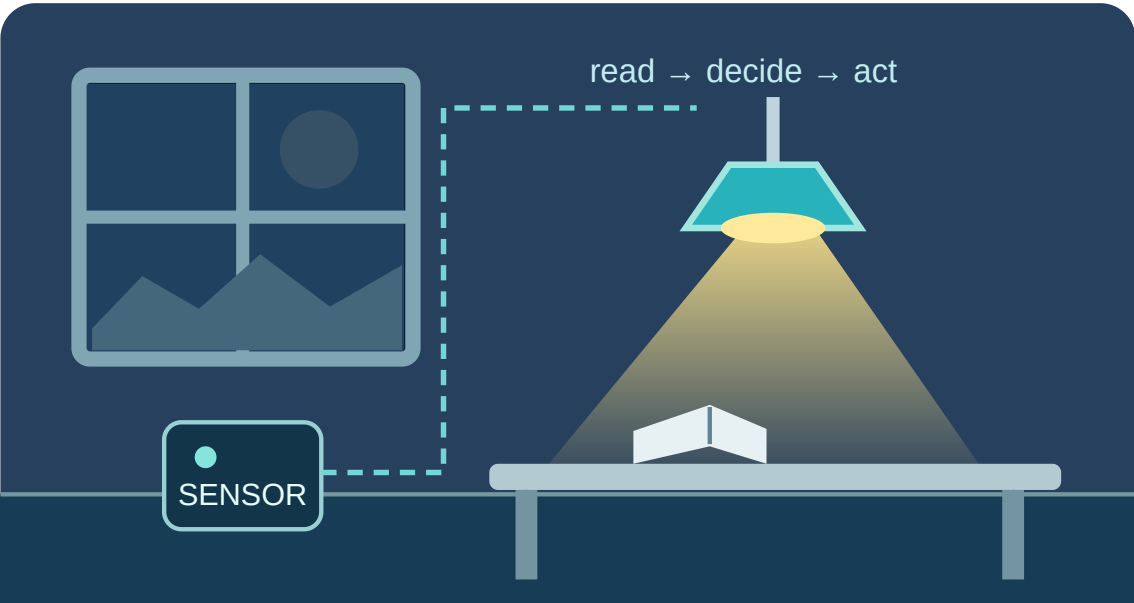
Study how systems sense, decide and act reliably.

My path grew one question at a time.

Your first question can come from a game, a sport, art, biology or everyday life.



Give the lamp a rule. Change its world.



Ambient brightness

0 · dark

100 · bright

10

Our instruction

Below 30 → turn ON
30 or above → turn OFF

It is below 30. The lamp turns on.

[Build this with blocks ↗](#)

INPUT

10

CONDITION

10 < 30: true

OUTPUT

Lamp ON

A program turns a goal into precise instructions. Here: read brightness → decide → switch the lamp. Simulated scale: 0–100.



YOU ALREADY KNOW THE IDEAS

09–10 MIN

Order. Choices. Repetition.

Sequence

Follow steps in order.

Read → decide → act.

Condition

Choose using a rule.

If it is dark, turn on a lamp.

Loop

Repeat a set of steps.

Check the next reading.

Data gives the rules something to work on: a name, a number, a list.



A readable bridge from a question to evidence.

READ A SMALL RULE

```
if brightness < 30:  
    print("Lamp ON")
```

A precise condition you can explain.

01 · Run an idea

Change one input. See an output.

02 · Work with evidence

NumPy arrays · pandas tables

03 · Compare and explain

Matplotlib charts · scikit-learn models

04 · Keep the whole story

Jupyter: code + results + explanation

Python call



NumPy library



Compiled calculations



A result to check

Python is a general-purpose language with scientific tools that fit our course. Libraries extend it; they are not separate languages. [Python tutorial](#) · [NumPy](#)



Start with the job you want to do.

- Investigate data
- Build a website
- Control a robot
- Create an app
- Animate a story

Python
General-purpose + scientific tools

JavaScript / TypeScript
Browser behavior + type checking

C / C++
Resources + hardware

Java / C#
Application ecosystems

SQL
Relational database queries

R
Statistics + graphics

Scratch / Blockly
Creative coding / block editors

Turn records into an answer

SQL selects records. Python or R can analyze them, build models and draw charts.

Database

↓

Table

↓

Model + chart

↓

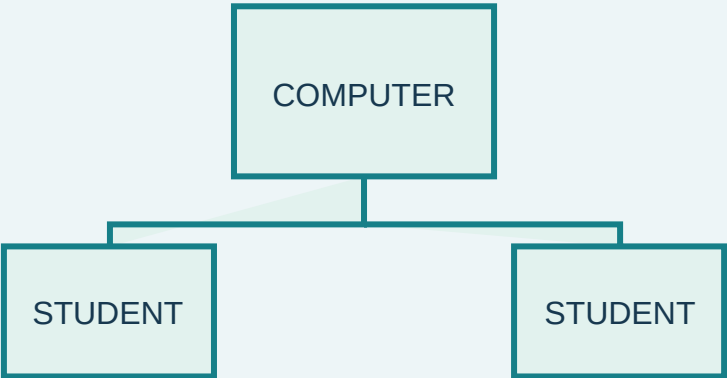
Evidence

Representative combinations; many choices overlap. A project can use several languages. [Python](#) · [JavaScript](#) · [TypeScript](#) · [R](#) · [Blockly](#)



New tools let more people try an idea.

BASIC	1964	Visual Basic	1991	Python	1989 → 1991
-------	------	--------------	------	--------	-------------



Who gets to use a computer?

At Dartmouth, BASIC and time-sharing opened computing to students across campus. Undergraduates helped build the system.

Programming could become something more students could try.

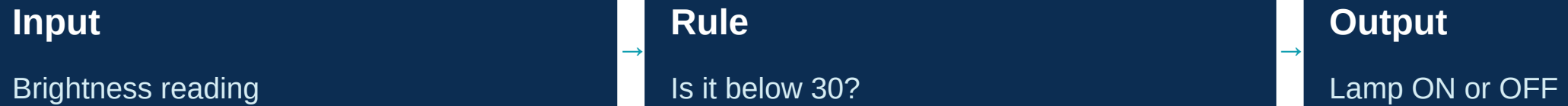
[Read the original history ↗](#)

Selected stories, not a complete timeline. Dates describe the languages, not Bo’s personal milestones.

BASIC (1964): broader student access. Visual Basic (1991): forms and events. Python (begun 1989; public 1991): practical tools, named after Monty Python.



A tiny automatic lamp.



Our specification: below 30 → ON · 30 or above → OFF

A simulated scale: 0 = dark, 100 = bright. These are teaching values, not a real sensor standard.



Can your partner run your instructions?



Student: write four commands.
Partner: execute exactly; do not guess.

Allowed: **E** = one cell east;
N = one cell north.

Start A1 → goal C3. Stay on the board; stop at a wall. E = east, N = north. [Try the same route with blocks ↗](#)



One program. Two maps. What changes?

Your program

Original map

Wall: B2

A3	B3	C3 GOAL
A2	B2 WALL	C2
A1 • HERE	B1	C1

Ready at A1

Changed map

Wall: C2

A3	B3	C3 GOAL
A2	B2	C2 WALL
A1 • HERE	B1	C1

Ready at A1

E = east · N = north · stop at a wall

Edit the route with blocks ↗

E E N N: original map reaches C3; changed map stops at C1 on command 3.
N N E E: both maps reach C3. Open the slide online to change and rerun the program.



The tools changed. Our goals can grow.

Learning task	Before AI assistants	With AI assistants
Get a first version	Write it; search books, docs and examples	Also ask AI for a small draft
Get unstuck	Trace code; ask teachers or peers	Also ask AI for a hint or explanation
Explore an idea	Build and revise a prototype	Try more candidate approaches
Know it works	Understand, test and explain	Understand, test and explain

These activities overlap across eras. AI adds options; it does not make understanding automatic.



WHY LEARN THE FOUNDATIONS?

24–25 MIN

Learn enough to ask better questions.

Read

Trace a short program.
Know what its inputs and outputs
mean.

Change

Make one intentional edit.
Predict its effect.

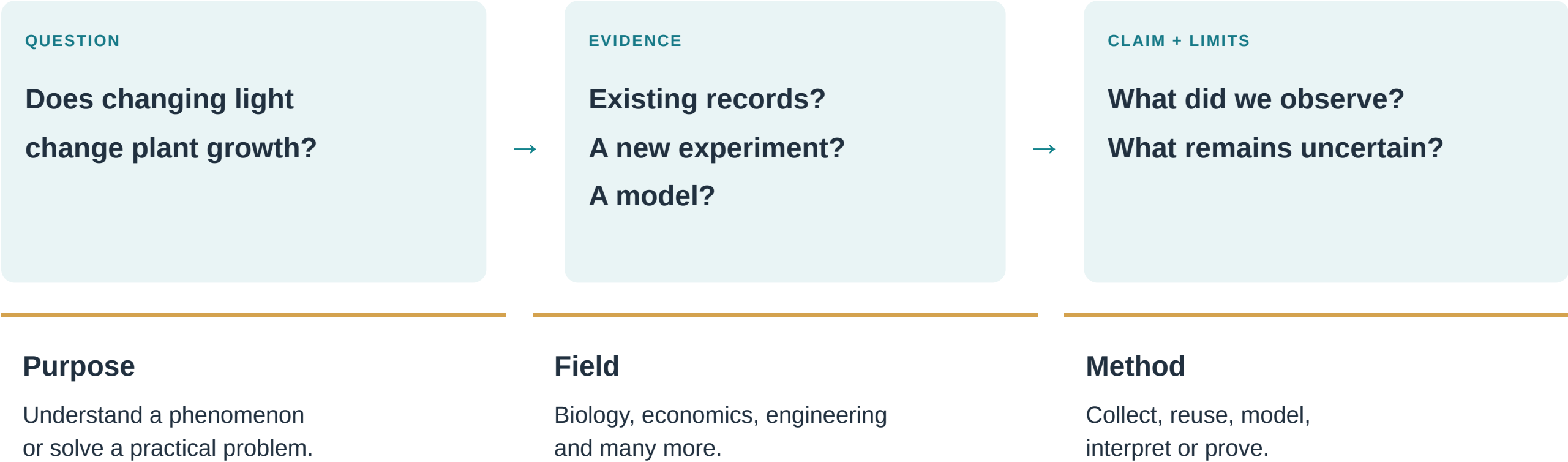
Judge

Choose a test.
Explain a result and its limits.

Useful in science, engineering, business, art—and in projects you invent.



A question needs evidence, not just an answer.

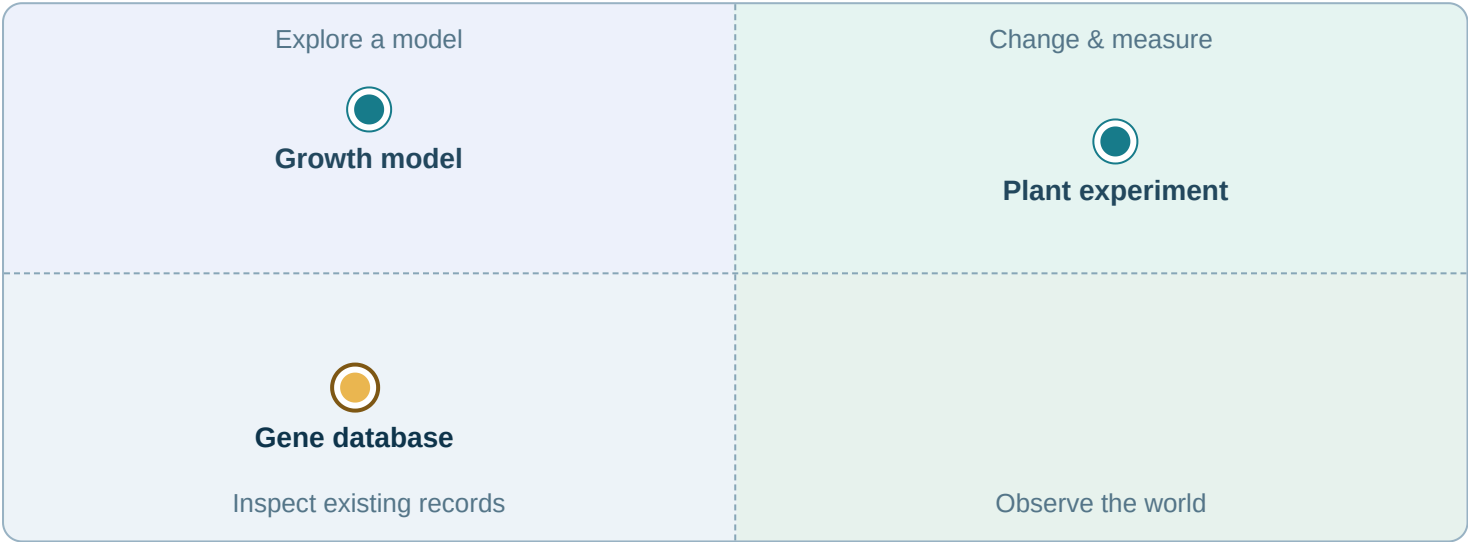


These are different ways to describe research. The next map focuses on empirical and computational examples; it does not cover every kind of research.



Your question decides what evidence you need.

↑ Test changes / ask “what if?”



↓ Describe / compare observations

← Reuse records / models

Collect new measurements →

Horizontal axis: need for new real-world data

Examples, not a census. Positions and marker sizes do not represent percentages. Projects can combine methods.

Same field, different evidence: biology can compare a gene database, model growth, or measure plants under changed light. Finance can use economic records, modeled scenarios, or experiments with participants. Environment can use satellite archives, scenarios, or field sensors. Robotics can use past logs, simulated routes, or real robot tests.

EXISTING DATA

Compare gene-expression groups

Do the recorded groups show different patterns?

EVIDENCE

Existing gene-expression records, such as NCBI GEO.

WHAT PYTHON HELPS US DO

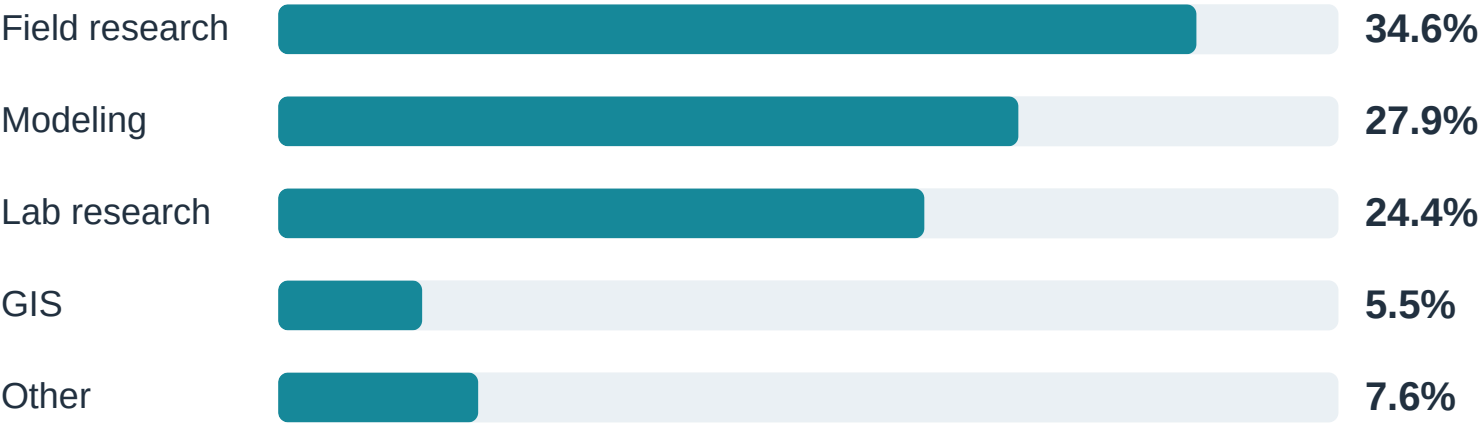
Read tables, compare groups, plot differences.

| A pattern in existing data does not by itself show a cause.



What did these respondents mainly do?

2017–18 volunteer survey · primary research activity · n = 2,032



A DIFFERENT QUESTION

38.3%

reported **frequent or constant** use of data collected outside their immediate team.

Same survey · n = 1,795
Some or all data reused.

Survey respondents, not all research. This voluntary sample is not a global census. Real projects can combine methods.

GIS = geographic information systems. Modeling is not the same as Digital Twin; data reuse is not database-only research. [Tenopir et al., PLOS ONE \(2020\), Figs. 3 & 8](#)

What makes a model a digital twin?

A particular physical system



Example reading: 24.0°C



A computer model

Proposed vent opening

25%

Starts from a chosen 24.0°C

Toy model's next estimate

25.0°C

$\text{start} + 2 - 0.04 \times \text{vent opening}$

A simulation can explore “what if?” without a connection to a real greenhouse.

A digital twin keeps a model connected to a particular real system through updated data. It still needs validation for its intended use.

Teaching diagram only: readings are synthetic, the formula is invented, and no hardware is connected. This is not a real digital twin.

With no data link, a new example reading does not update the model. With the illustrated link enabled, the model starts from the latest example reading. A data connection alone does not establish a validated digital twin.



AI can help. You make the decisions.

Ask AI to contribute

- Explain an unfamiliar line.
- Suggest a small implementation.
- Offer a test or a different approach.

The learner stays responsible

- Define the goal and inputs.
- Run and inspect the result.
- Accept, revise or reject a suggestion.

Generated code may be incorrect or miss your intent. Review and testing still matter. [Source: GitHub Docs](#)



Ask for help that keeps you thinking.

“Here is my rule and my prediction.
Ask me one question that could expose a mistake.
Give me a hint, then wait for my test.”

Before asking

Write what you expect and why.

After asking

Test one suggestion. Keep your decision.

An instructor can demonstrate AI. No student AI account is needed today.



What would count as correct?

Requirement: brightness below 30 → ON; 30 or above → OFF.

Input	Expected output	Why test it?
10	Write your prediction.	A darker reading
50	Write your prediction.	A brighter reading
30	Write your prediction.	Exactly at the boundary

Student: predict. Partner: check against the requirement. Then switch roles.



ACTIVITY 2 · REAL PYTHON · 36-41 MIN

Run it. Find the mismatch. Repair it.

Classroom-made draft simulating an AI suggestion · not a real AI transcript.

Change brightness to **10**, **50**, then **30**. Compare with your predictions. Repair one symbol and rerun all three.

```
1 # Simulated brightness: 0 = dark, 100 = bright
2 brightness = 10
3 if brightness <= 30:
4     print("Lamp ON")
5 else:
6     print("Lamp OFF")
```



Run



Explain each line



Explain



Debug



Improve



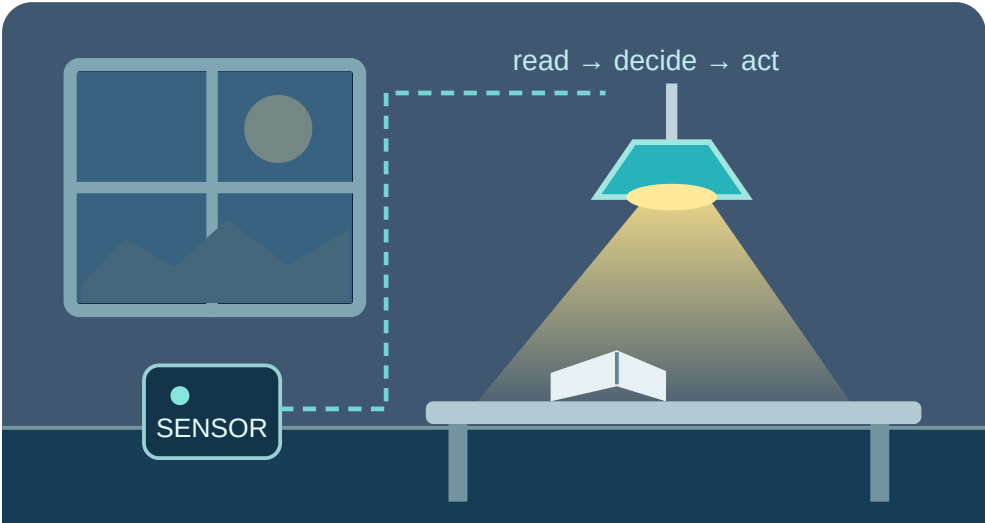
Mira

Python loads on first Run

► Run the code to see the output here.



Test the rule. See the mismatch.



INPUT	CONDITION	OUTPUT
30	$30 \leq 30$: true	Lamp ON

Draft ≤ 30 : 10 ON · 50 OFF · 30 ON (fails) · 0 ON · 100 OFF.
Repair < 30 : 10 ON · 50 OFF · 30 OFF · 0 ON · 100 OFF.

Requirement: below 30 → ON; 30 or above → OFF. Rule simulation; compare with the [real Python experiment](#).

undefined online

Ambient brightness

0 · dark

100 · bright

30

Test this rule

Draft: brightness ≤ 30

Mismatch: at 30, the requirement says OFF; this rule gives ON.

[Build this with blocks ↗](#)



HOW WE TEACH

43–45 MIN

You just experienced our method.

Predict

Commit to an expectation



Run + inspect

Trace code and results



Modify

Change one thing



Test + explain

Keep evidence and limits

Then make a small version of your own. Ask AI for focused help along the way.

Adapted from PRIMM: Predict, Run, Investigate, Modify, Make. We add explicit AI decisions and test records. [Raspberry Pi Foundation](#)



WHAT A LESSON LOOKS LIKE

45–46 MIN

Small programs. Visible behavior. Your voice.

See the idea

A short Python example.
A grid, chart or value display.

Try one change

Predict before pressing Run.
Inspect one line at a time.

Explain your choice

Keep code and results.
Describe a fix, a test and a limit.

[Class 01: Your First Python Experiment ↗](#) · [Python in Motion ↗](#)



Fifteen lessons. One growing research story.

 CLASSES 1–3

Build the language

Read a rule before asking a model to learn one.

- 1 Your First Python Experiment
- 2 Decisions & Loops
- 3 Functions & Research Records

YOU KEEP

A short program you can explain

 CLASSES 4–8

Build the evidence

Understand data before judging a prediction.

- 4 NumPy & Simulation
- 5 pandas & Tables
- 6 Cleaning With a Reason
- 7 Charts That Answer Questions
- 8 Exploring a Research Question

YOU KEEP

A question, data, and a test plan

 CLASSES 9–13

Compare fairly

Keep the test fixed while the method changes.

- 9 Your First Machine-Learning Baseline
- 10 Predicting Numbers
- 11 Predicting Categories
- 12 Trees & Overfitting
- 13 A Fair Model Comparison

YOU KEEP

A baseline, actual results, and limits

 CLASSES 14–15

Explain and share

A result becomes useful when others can check it.

- 14 A Reproducible Research Story
- 15 Project Showcase

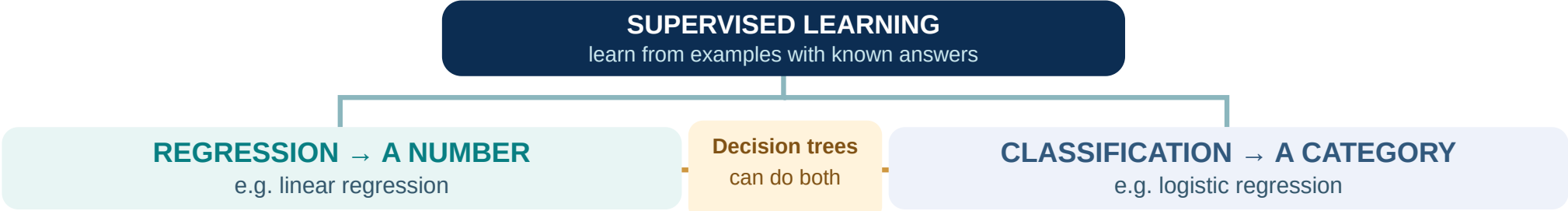
YOU KEEP

A runnable project and a clear story

One small project grows across the course. Class 0 is separate. [Explore the 15-lesson plan ↗](#)



Where do the later four lessons fit?



Predicting numbers 10	Predicting categories 11	Trees & overfitting 12	A fair comparison 13
TASK + METHOD	TASK + DECISION	MODEL + COMPLEXITY	EVALUATION
Linear regression	Classifiers + thresholds	Decision trees	Same held-out test
Estimate a temperature; inspect the error.	Find a missed detection and a false alarm.	Trace branches; compare shallow and deep.	Compare the chosen method with a baseline.

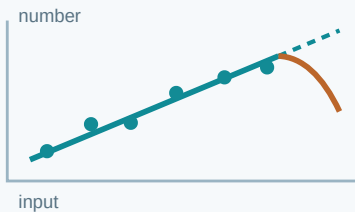
Four lessons, different jobs. Regression and classification are tasks. A baseline is a comparison role. Evaluation checks a method.

A partial map of classical ML. Logistic regression is an example for orientation; the core plan names classification. [Method families](#) · [Evaluation](#)

Small models make important ideas visible.

A line you can inspect

Linear regression · Class 10



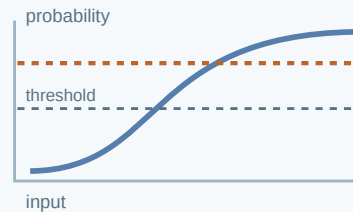
See how an input relates to a numerical prediction.

TEST NEXT

Check curvature, unusual points, and extrapolation.

A decision you can discuss

Logistic regression · category example



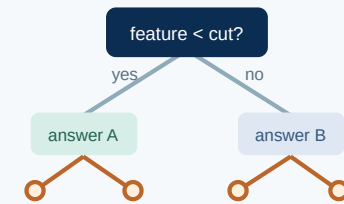
Separate a probability estimate from the final yes/no threshold.

TEST NEXT

Check misses, false alarms, and the threshold trade-off.

A rule you can follow

Decision trees · Class 12



Trace the path; see how adding branches changes complexity.

TEST NEXT

Check whether extra depth helps on validation data.

Why we start here: small examples, visible assumptions, and checkable mistakes.

These are established model families. We select them for the ideas they reveal. Project choice depends on the question and evidence.

Schematic illustrations, not fitted models or measured results. [Linear models](#) · [Logistic example](#) · [Trees](#)



Be the curious audience.

Ask about thinking

“What did you expect?”

“What surprised you?”

“What would you test next?”

Make room for ownership

Let the student make the edit.

Ask them to explain an AI suggestion.

Notice a thoughtful correction.

A useful progress signal: “I found why it failed, and here is how I checked the fix.”

[中文家庭课程指南](#) ↗



What would help you begin?

“I have never coded.”

Start with reading and changing a small example.

“Will we still write code?”

Yes. We read, edit, debug and build, with AI support.

“How will we see progress?”

Look for predictions, independent explanations and checked revisions.

Your questions · 3 minutes



TAKE ONE QUESTION WITH YOU

57–60 MIN

Your first project starts with curiosity.

Student · explain

Why did brightness 30 expose the mistake?

Student · imagine

“I want to build or explore ____.”
“My first test would be ____.”

Partner · reflect

Name one decision the student made today.

[Next: Class 01 · Your First Python Experiment ↗](#)

[Print the activity sheet](#) · [中文讲者备注](#) · [Download slides PDF](#)



OPTIONAL · AFTER THE 60-MINUTE SESSION

REFERENCE

For students, families and instructors.

Language choices & history

[Annotated source notes](#) · [Python](#) · [BASIC](#) · [Visual Basic](#)

Research methods & data

[Sources and statistical scope](#) · [GEO](#) · [FRED](#) · [NASA Earthdata](#)

Models & measurement

[NIST digital twins](#) · [Tenopir survey, Figs. 3 & 8](#)

Our curriculum & teaching

[15-lesson plan and five project routes](#) · [Supervised methods](#) · [Computing pedagogy](#)

Sources reviewed 12 September 2026. Biography supplied by the instructor. Our diagrams and toy examples are teaching designs; the survey has its own stated population and dates.